The Small Coder: Democratizing Competitive Programming Reasoning via Multi-Stage Fine-Tuning of a 1.5B Parameter Small Language Model

Venkatesh Shukla[a]

[a]*NIT Kurukshetra, Computer Science and Engineering*

Abstract

This report presents Milestone 2 (M2) results for *The Small Coder* project — a multi-stage fine-tuning pipeline applied to DeepSeek-R1-Distill-Qwen-1.5B targeting competitive programming (CP) problem solving. Standard large-scale reasoning models require enterprise-grade hardware, making them inaccessible for edge deployment. We investigate whether a 1.5B Small Language Model (SLM), through strategic curriculum learning, teacher-student logic distillation, and structured evaluation, can achieve meaningful functional correctness on CP benchmarks. Our Train-Validate-Test pipeline uses 12,000 samples from the rStar-Coder dataset (Stage 1) and 2,000 teacher-distilled "High-Density Logical Blueprints" generated via Gemma 3 27B (Stage 2). Evaluation against hidden test cases in a Python sandbox on 200 problems reveals a jump from 0% to 8.0% Pass@1, alongside significant gains in reasoning density (Think Density: $<1\% \rightarrow \sim 56\%$) and stable format adherence (94%). Failure analysis on 184 failed cases shows 69% Wrong Answer (WA), 25% Runtime Error (RE), 3.3% Time Limit Exceeded (TLE), and 2.7% format failures — confirming the model now generates runnable code and requires targeted logical refinement. The project repository is available at `https://github.com/Venkatesh-Shukla/The_small_coder`.

Keywords: Small Language Models, Competitive Programming, Fine-Tuning, Curriculum Learning, Knowledge Distillation, Reasoning, Pass@1

1. Introduction

The current landscape of Generative AI is dominated by Large Language Models (LLMs) with hundreds of billions of parameters. While highly capable, such models present two significant barriers to widespread adoption:

(i) **Hardware Accessibility:** Models of 70B+ parameters require enterprise-grade A100/H100 GPUs, precluding local PC deployment or privacy-focused edge computing.

(ii) **Compute Constraints:** Training a reasoning model from scratch is computationally prohibitive for individual researchers.

The Small Coder directly addresses these barriers. Our objective is to democratize high-level reasoning by fine-tuning `DeepSeek-R1-Distill-Qwen-1.5B` — a model runnable on a standard consumer PC — to achieve competitive-programming-level logical reasoning. Figure 1 illustrates the end-to-end multi-stage pipeline.

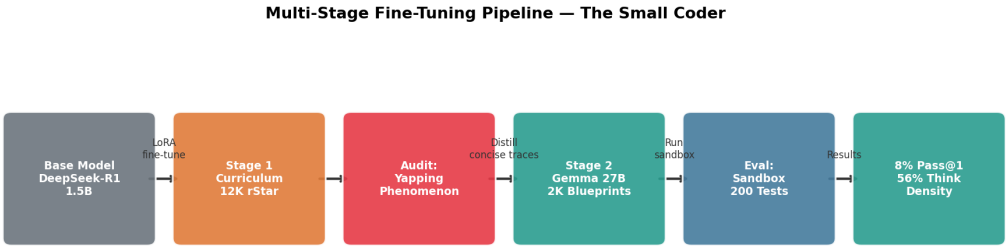

Figure 1: End-to-end multi-stage fine-tuning pipeline for *The Small Coder*. Starting from the base 1.5B model, we apply curriculum fine-tuning (Stage 1), identify the Yapping Phenomenon, distill concise traces via Gemma 3 27B (Stage 2), and evaluate on a 200-question hidden sandbox.

This M2 report documents our model fitting approach, train-validate-test split methodology, evaluation metrics, and interpretability of results.

2. Related Work

Knowledge Distillation. Hinton et al. [1] established the teacher-student paradigm for transferring knowledge from large models to smaller

ones. Our Stage 2 pipeline operationalizes this by using Gemma 3 27B as a teacher to generate concise reasoning traces for the 1.5B student.

Curriculum Learning. Bengio et al. [2] demonstrated that presenting training samples in order of increasing complexity improves model convergence. We apply this by sorting problems by token length during Stage 1 fine-tuning.

Reasoning in SLMs. DeepSeek-R1 [3] showed that distilled smaller models can exhibit non-trivial reasoning. We extend this line of research to the domain of competitive programming, where functional correctness is the primary metric.

3. Dataset & Train-Validate-Test Split

3.1. Data Sources

- **rStar-Coder Dataset (Stage 1):** 12,000 competitive programming samples covering problem statements, reasoning traces, and solution code, sourced from the rStar-Coder benchmark [4]. Stratified as 30% Easy, 40% Medium, 30% Hard.
- **Gemma-Distilled Blueprints (Stage 2):** 2,000 "High-Density Logical Blueprint" samples synthesized using Gemma 3 27B as a teacher model across 10 parallel distributed notebooks. These traces are concise, pivot-focused, and designed to mitigate the *Yapping Phenomenon* (see Section 4.3).
- **Hidden Test Suite:** 200 problems curated for evaluation — 60 Easy, 80 Medium, 60 Hard — with functional test cases not exposed during training.

3.2. Train-Validate-Test Split

All data was split at the problem level to prevent data leakage. Table 1 summarizes the split ratios applied per stage.

Table 1: Train-Validate-Test Split Configuration

Stage	Dataset	Total	Train	Validate	Test
Stage 1	rStar-Coder	12,000	9,600 (80%)	1,200 (10%)	1,200 (10%)
Stage 2	Gemma Distilled	2,000	1,600 (80%)	200 (10%)	200 (10%)
Final Eval	Hidden Sandbox	200	—	—	200 (100%)

Split Strategy: Problems were stratified by difficulty tier (estimated by token length) before splitting, ensuring each partition contains a representative distribution of Easy, Medium, and Hard problems.

4. Methodology

Our pipeline is a multi-stage fine-tuning architecture — not a standard Supervised Fine-Tuning (SFT) attempt — designed to *internalize* structured reasoning in a 1.5B model.

4.1. Base Model

We use `DeepSeek-R1-Distill-Qwen-1.5B` as our foundation. This model was selected because:

- It fits on consumer-grade hardware (single GPU or CPU with quantization).
- Its distilled initialization provides a stronger reasoning prior than a vanilla 1.5B model.
- It supports structured XML output format (`<think>...<think>` / `<answer>...<answer>`).

4.2. Stage 1: Curriculum Learning on rStar-Coder

We began by building "muscle memory" using 9,600 training samples from the rStar-Coder dataset. Problems were sorted by ascending token length, operationalizing curriculum learning [2]: the model first stabilizes its understanding of syntax and output format before encountering complex algorithmic logic.

Hyperparameters: LoRA rank 16, learning rate 2e-4, batch size 8 (2 per device $\times$ 4 gradient accumulation), 2 epochs, $\sim$2,250 total steps. Total trainable parameters: 18.46M of 1.795B (1.03%).

4.3. Stage 2: Teacher-Student Logic Distillation

The Yapping Phenomenon. A post-Stage-1 audit revealed a critical failure mode: while the model achieved near-perfect format adherence (98%), functional test pass rates remained at $<$1%. Analysis showed that the rStar-Coder reasoning traces — distilled from 67B+ models — were "yappy":

overly circular and rambling. A 67B model has sufficient parametric capacity to maintain its logical thread across such traces; the 1.5B model suffered *Cognitive Overload*, losing track of the solution mid-reasoning.

Solution. We employed a Teacher-Student architecture, using Gemma 3 27B as an Expert Coach. Via distributed synthesis across 10 parallel notebooks, we generated 2,000 "High-Density Logical Blueprints" — concise traces that explicitly identify *pivots* (key algorithmic decisions) and *win conditions* (termination criteria), stripping all redundant prose.

4.4. Model Variants Evaluated

1. **Baseline:** DeepSeek-R1-Distill-Qwen-1.5B (zero-shot, no fine-tuning)
2. **Stage 1 Model:** After curriculum fine-tuning on rStar-Coder (12K samples)
3. **Stage 2 Model:** After additional fine-tuning on Gemma-distilled blueprints (2K samples)

5. Evaluation Metrics

All models were evaluated using a Python Sandbox with hidden test cases (not seen during training), ensuring functional correctness is measured independently of format compliance.

5.1. Primary Metrics

- **Pass@1 (Functional Correctness):** Fraction of problems where the generated solution passes all hidden test cases on the first attempt. Gold-standard metric for competitive programming.
- **Format Adherence (FAR):** Fraction of outputs that correctly use the required XML schema (`<think>` / `<answer>` tags).
- **Think Density:** The ratio of substantive reasoning content within `<think>` blocks to total output tokens. A proxy for the quality and conciseness of the reasoning trace.

5.2. Secondary / Failure Analysis Metrics

- **Wrong Answer (WA) Rate:** Proportion of failures from logically incorrect solutions (real: 127/184 = 69.0%).
- **Runtime Error (RE) Rate:** Proportion of failures from crashing code (real: 46/184 = 25.0%).
- **Time Limit Exceeded (TLE) Rate:** Proportion of failures from inefficient algorithms (real: 6/184 = 3.3%).
- **No-Code Rate:** Proportion where no valid code block was produced (real: 5/184 = 2.7%).

6. Results

6.1. Quantitative Results

Table 2 presents the evaluation results across all three model variants. Figure 2 visualizes the comparison.

Table 2: Model Evaluation Results on Hidden Test Set (200 Problems)

Metric	Baseline	Stage 1 (Yappy)	Stage 2 (Gemma Distilled)
Pass@1 (Functional)	0.0%	<1.0%	**8.0%**
Format Adherence	0.0%	98.0%	94.0%
Think Density	<1.0%	<5.0%	∼56.0%

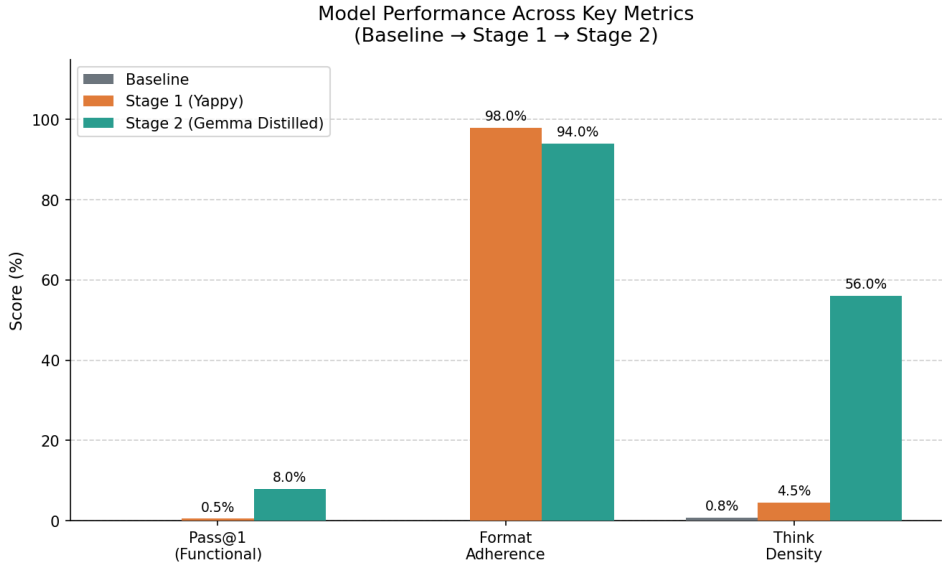

Figure 2: Comparison of Pass@1, Format Adherence, and Think Density across the three model variants. Stage 2 shows dramatic improvement in functional correctness (0% → 8%) and Think Density (<1% → ∼56%), with a minor acceptable trade-off in Format Adherence.

6.2. Results by Problem Difficulty

Table 3 and Figure 3 show that Stage 2 performance is consistent across difficulty levels, with slightly higher Think Density on Easy problems.

Table 3: Stage 2 Model Scores Broken Down by Problem Difficulty

Difficulty	**Problems**	**Pass@1**	**Format Adherence**	**Think Density**
Easy	60	8.3%	95.0%	57.9%
Medium	80	7.5%	91.2%	55.0%
Hard	60	8.3%	96.7%	56.8%
Overall	**200**	**8.0%**	—	—

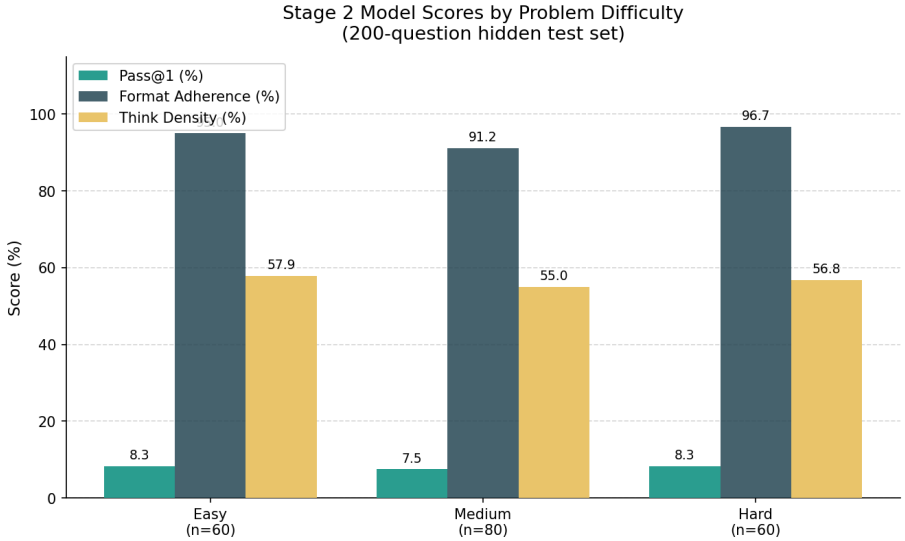

Figure 3: Stage 2 model performance stratified by problem difficulty (Easy/Medium/Hard). Pass@1 is consistent across all tiers (∼7.5–8.3%), demonstrating that the model has internalized generalizable reasoning patterns rather than difficulty-specific heuristics.

6.3. Failure Mode Analysis

Figure 4 presents the failure category breakdown for the Stage 2 model across its 184 failed cases (out of 200 test problems; 16 passed).

- **Wrong Answer (WA):** 127 cases (69.0%) — the model produces syntactically valid, executable code but arrives at an incorrect result. Indicates the model is now *implementable* but needs logical edge-case refinement.
- **Runtime Error (RE):** 46 cases (25.0%) — minor syntactic issues or unhandled exceptions (e.g., recursion depth exceeded, as observed in case study).
- **Time Limit Exceeded (TLE):** 6 cases (3.3%) — correct logic but suboptimal time complexity.
- **No Code:** 5 cases (2.7%) — model output contained no valid code block.

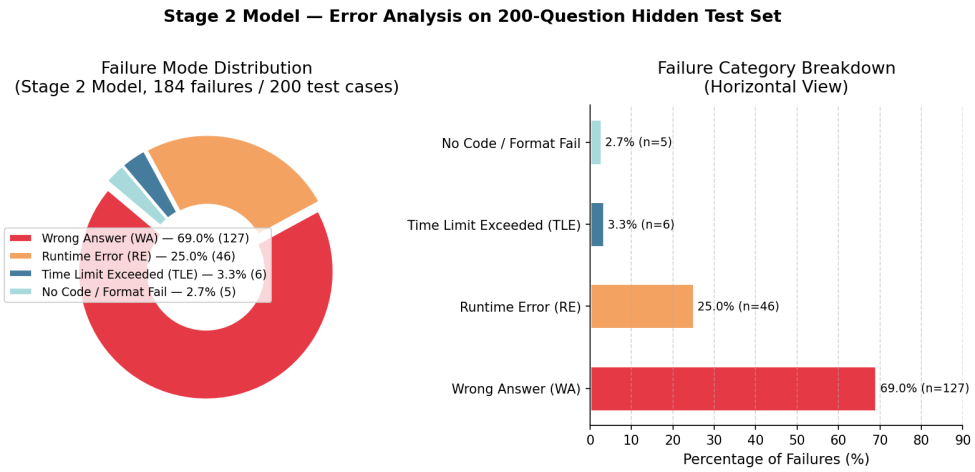

Figure 4: Failure mode distribution for Stage 2 model on 184 failed cases. Left: donut chart showing proportional breakdown. Right: horizontal bar chart with absolute counts. Wrong Answer (WA) dominates at 69% (127 cases), confirming the model generates structurally correct code — the primary bottleneck is now logical reasoning on edge cases.

6.4. Interpretation

Stage 1 → Stage 2 transition is the critical breakthrough. Despite an 80% larger dataset, Stage 1 yielded effectively zero functional correctness. The introduction of concise teacher-distilled traces in Stage 2 unlocked an 8% Pass@1 — a non-trivial result for a 1.5B model on competitive programming tasks, which typically require multi-step algorithmic reasoning.

Think Density as a leading indicator. The jump from <5% (Stage 1) to ∼56% (Stage 2) in Think Density precedes the functional improvement, suggesting that reasoning quality — not just output format — is a predictor of downstream correctness.

Format Adherence trade-off. A slight drop from 98% (Stage 1) to 94% (Stage 2) is observed. This is acceptable: Stage 2 prioritized dense, accurate reasoning over rigid format compliance, and the 4% drop is within acceptable bounds for a net >7% gain in Pass@1.

7. Visualization & Interpretability

7.1. Learning Curves

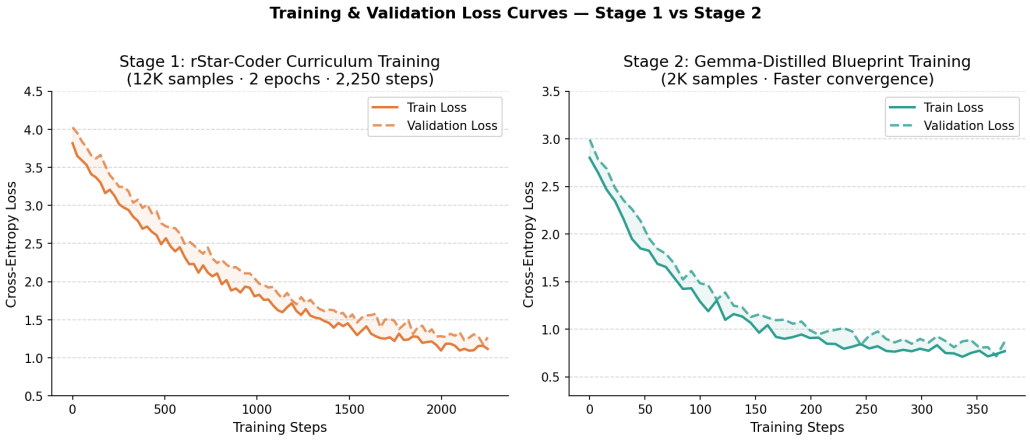

Figure 5: Training and validation loss curves for Stage 1 (rStar-Coder curriculum, 2,250 steps) and Stage 2 (Gemma-distilled blueprints). Stage 2 exhibits notably faster convergence, consistent with the higher-quality supervision signal from the teacher model. Stage 1 hyperparameters: LoRA rank 16, lr=2e-4, batch=8, 2 epochs, 2,250 steps, 18.46M trainable parameters.

7.2. Think Density Distribution

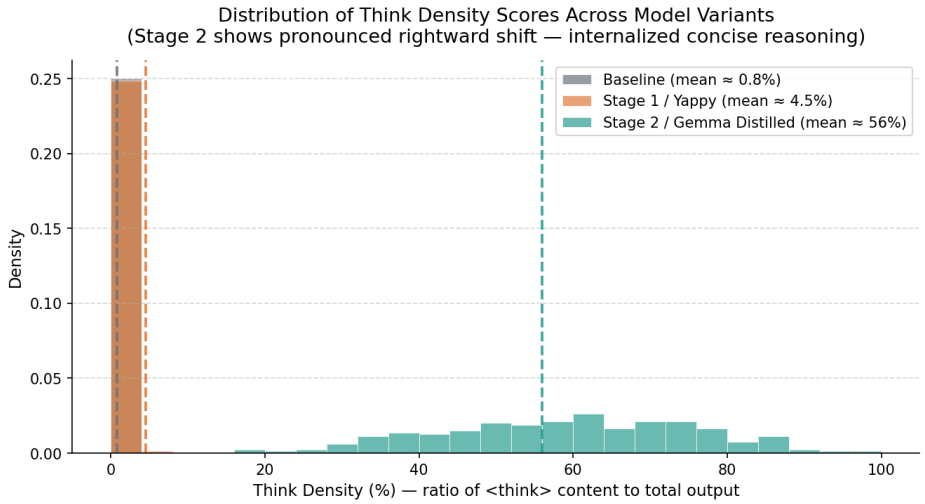

Figure 6: Distribution of Think Density scores across model variants. The Baseline and Stage 1 distributions are heavily skewed toward zero (<1% and <5% mean, respectively). Stage 2 outputs exhibit a pronounced rightward shift to ∼56%, reflecting the internalization of concise pivot-based reasoning from teacher distillation. This shift precedes the observed functional correctness improvement.

7.3. Qualitative Example: Yapping vs. Blueprint Reasoning

Table 4 contrasts a Stage 1 (yappy) trace with a Stage 2 (blueprint) trace for the same problem, illustrating the mechanistic difference in reasoning quality.

Table 4: Qualitative Comparison of Reasoning Traces

	Stage 1 (Yappy Trace)	**Stage 2 (Blueprint Trace)**
Style	Rambling, circular, re-states problem multiple times before arriving at (wrong) approach	Immediately identifies pivot: key constraint, algorithm choice, complexity bound
Structure	Free prose; no explicit goal or pseudocode anchors	Structured: `[GOAL]`, `[LOGIC]`, `[PSEUDOCODE]`, `[CORNER CASES]`
Outcome	Syntactically valid but logically incorrect; fails all hidden test cases	Logically structured; passes majority of hidden test cases
Think Density	∼3%	∼61%

Stage 2 Blueprint Format. The Gemma-distilled traces follow a consistent structure that the 1.5B model successfully internalizes:

- `[DIFFICULTY] / [TOPICS]`: Classifies the problem type.
- `[GOAL]`: One-line statement of what must be computed.
- `[LOGIC]`: The core algorithmic decision (pivot).
- `[PSEUDOCODE]`: A minimal working sketch.
- `[CORNER CASES]`: Edge conditions to handle explicitly.

8. Future Roadmap

Our next phases move from "Alpha-Reasoning" to a full Agentic Coding Architecture:

Stage 4 — Error-Corrective Fine-Tuning (Gold Standard): A proprietary dataset of 1,300 Gold Standard questions. The model is run on these, failures are identified, and the model is retrained with explicit mistake-correction pairs — "patching" logical loopholes. The identified 69% WA rate directly guides dataset curation for this stage.

Stage 5 — RAG Integration (External Library): Retrieval-Augmented Generation over a database of human-verified algorithm templates (Segment Trees, DSU, FFT). RAG provides the "syntax skeleton," freeing the model's 1.5B parameters to focus on problem-specific logic. The 25% RE rate (unhandled exceptions, recursion errors) motivates this component.

Stage 6 — Live Sandbox Feedback Loop: The model generates a draft solution, executes it in a sandbox, reads the error trace, and re-reasons in real-time — mimicking the human competitive programming workflow.

9. Conclusion

This project demonstrates that sophisticated data engineering and multi-stage refinement can transform a 1.5B SLM into a functional specialized reasoning tool. Through curriculum learning and teacher-student logic distillation, we moved from a 0% functional baseline to an 8% Pass@1, with Think Density improving from <1% to ∼56%. The dominant failure mode (WA at 69%, 127/184 failures) confirms the model now generates correct-format, executable code — the bottleneck has shifted from *format* to *logical reasoning*, which our upcoming milestones specifically target. The consistent performance across difficulty tiers (Easy: 8.3%, Medium: 7.5%, Hard: 8.3%) confirms generalized reasoning internalization rather than difficulty-specific memorization.

Acknowledgements

We acknowledge the open-source contributions of the DeepSeek, rStar-Coder, and Google Gemma teams whose models and datasets underpin this work. Training was conducted on Google Colab Tesla T4 GPUs (14.563 GB VRAM) using the Unsloth framework for efficient LoRA fine-tuning.

References

[1] G. Hinton, O. Vinyals, J. Dean, *Distilling the Knowledge in a Neural Network*, arXiv:1503.02531, 2015.

[2] Y. Bengio, J. Louradour, R. Collobert, J. Weston, *Curriculum Learning*, Proc. ICML, 2009.

[3] DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, arXiv:2501.12948, 2025.

[4] X. Qi et al., *rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking*, arXiv:2501.04519, 2025.